

ÁRVORE AVL

Problema do balanceamento

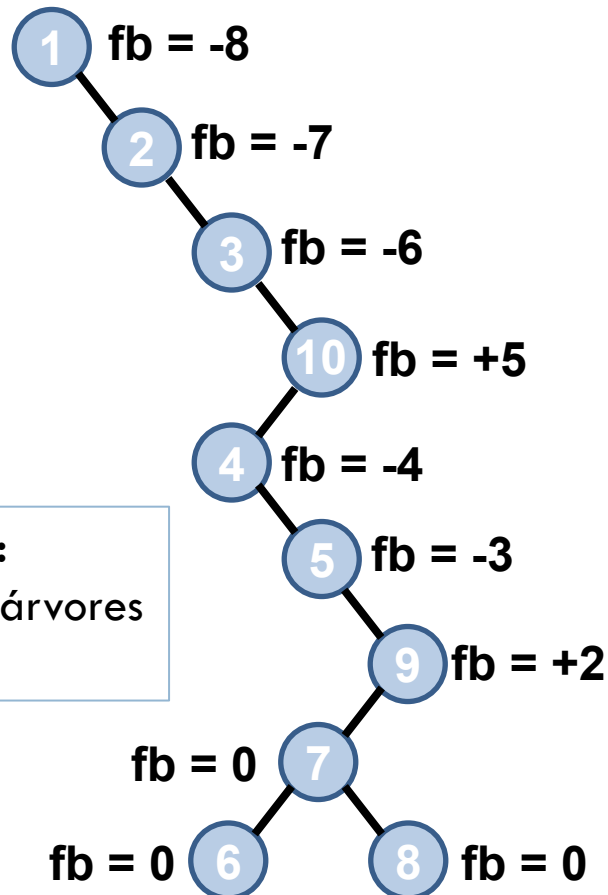
- A eficiência da busca em uma árvore binária depende do seu balanceamento.
 - ▣ $O(\log N)$, se a árvore está balanceada
 - ▣ $O(N)$, se a árvore não está balanceada
 - N corresponde ao número de nós na árvore

Problema do balanceamento

- Infelizmente, os algoritmos de inserção e remoção em árvores binárias não garantem que a árvore gerada a cada passo esteja balanceada.
- Dependendo da ordem em que os dados são inseridos na árvore, podemos criar uma árvore na forma de uma escada

Problema do balanceamento

- Inserção dos valores {1,2,3,10,4,5,9,7,8,6}



fb(fator de balanceamento):
diferença de altura das sub-árvores
de cada nó

Problema do balanceamento

- Solução para o problema de balanceamento
 - ▣ Modificar as operações de inserção e remoção de modo a balancear a árvore a cada nova inserção ou remoção.
 - Garantir que a diferença de alturas das sub-árvores esquerda e direita de cada nó seja de no máximo uma unidade
 - ▣ Exemplos de árvores balanceadas
 - Árvore AVL
 - Árvore 2-3-4
 - Árvore Rubro-Negra

Árvore AVL

□ Definição

- ▣ Tipo de árvore binária balanceada com relação a altura das suas sub-árvores
- ▣ Criada por **Adelson-Velskii** e **Landis**, motivo para a sua nomenclatura, em 1962

Árvore AVL

□ Definição

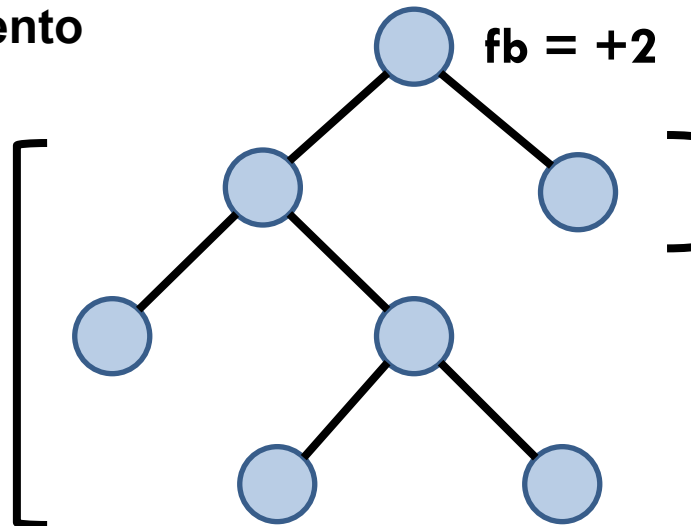
- ▣ Permite o rebalanceamento local da árvore
 - Apenas a parte afetada pela inserção ou remoção é rebalanceada
- ▣ Usa **rotações simples** ou **duplas** na etapa de rebalanceamento
 - Executadas a cada inserção ou remoção
 - As rotações buscam manter a árvore binária como uma árvore quase completa
 - Custo máximo de qualquer algoritmo é **$O(\log N)$**

Árvore AVL

- Objetivo das rotações:
 - ▣ Corrigir o **fator de balanceamento** (ou **fb**)
 - Diferença entre as alturas das sub-árvores de um nó
 - ▣ Caso uma das sub-árvores de um nó não existir, então a altura dessa sub-árvore será igual a -1.

Fator de Balanceamento
 $FB = AE - AD$

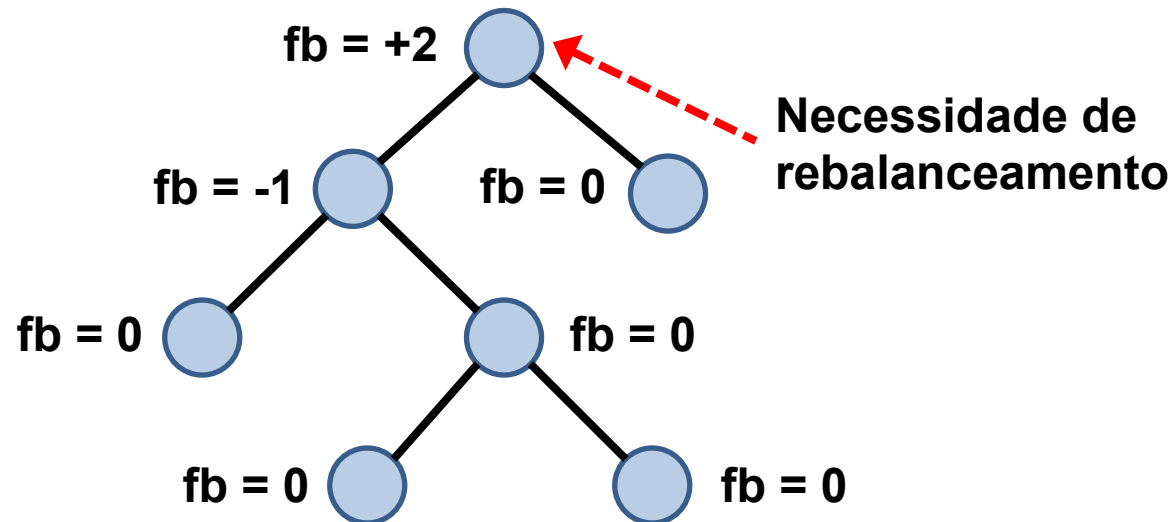
AE
Altura da
sub-árvore
ESQUERDA



AD
Altura da
sub-árvore
DIREITA

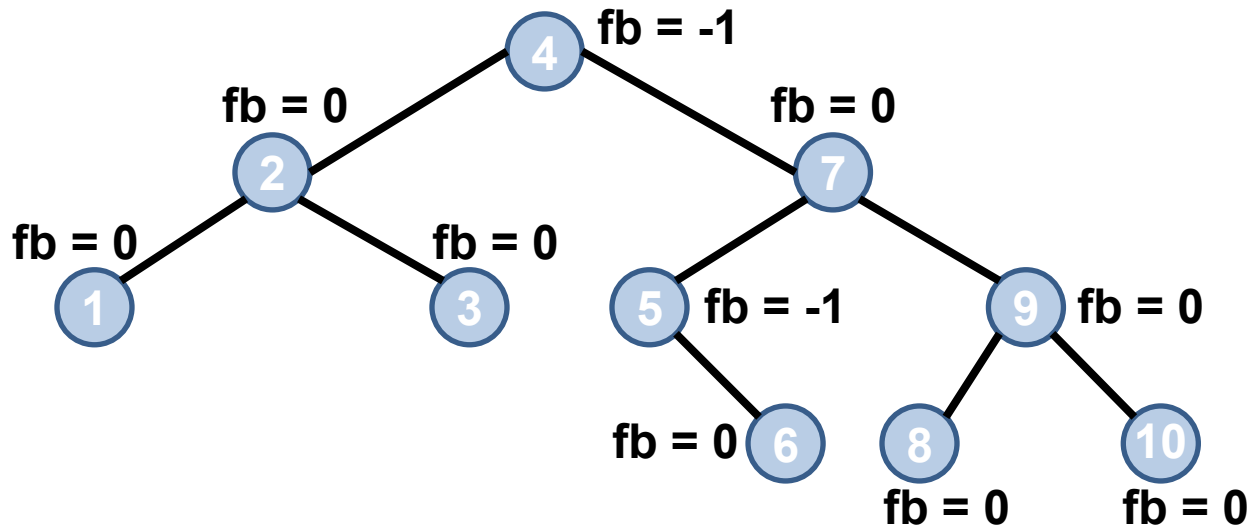
Árvore AVL

- As alturas das sub-árvores de cada nó diferem de no máximo uma unidade
 - ▣ O fator de balanceamento deve ser +1, 0 ou -1
 - ▣ Se **fb** > +1 ou **fb** < -1: a árvore deve ser balanceada naquele nó



Árvore AVL

- Voltando ao problema anterior
- Inserção dos valores {1,2,3,10,4,5,9,7,8,6}



TAD Árvore AVL

□ Definindo a árvore

▣ Criação e destruição: igual a da árvore binária

```
1 //Arquivo ArvoreAVL.h
2 typedef struct NO* ArvAVL;
3
4 //Arquivo ArvoreAVL.c
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include "ArvoreAVL.h" //inclui os Protótipos
8 struct NO{
9     int info;
10    int alt; //altura daquela sub-árvore
11    struct NO *esq;
12    struct NO *dir;
13 };
14 //programa principal
15 ArvAVL* raiz; //ponteiro para ponteiro
```

TAD Árvore AVL

□ Calculando o fator de balanceamento

```
1 //Funções auxiliares
2 //Calcula a altura de um nó
3 int alt_NO(struct NO* no) {
4     if(no == NULL)
5         return -1;
6     else
7         return no->alt;
8 }
9 //Calcula o fator de balanceamento de um nó
10 int fatorBalanceamento_NO(struct NO* no) {
11     return labs(alt_NO(no->esq) - alt_NO(no->dir));
12 }
```

Rotações

- Objetivo: corrigir o **fator de balanceamento** (ou **fb**) de cada nó
 - ▣ Operação básica para balancear uma árvore AVL
- Ao todo, existem dois tipos de rotação
 - ▣ Rotação simples
 - ▣ Rotação dupla

Rotações

- As rotações diferem entre si pelo sentido da inclinação entre o nó pai e filho
 - ▣ Rotação simples
 - O nó desbalanceado (pai), seu filho e o seu neto estão todos no mesmo sentido de inclinação
 - ▣ Rotação dupla
 - O nó desbalanceado (pai) e seu filho estão inclinados no sentido inverso ao neto
 - **Equivale a duas rotações simples.**

Rotações

- Ao todo, existem duas rotações simples e duas duplas:
 - ▣ Rotação simples a direita ou Rotação LL
 - ▣ Rotação simples a esquerda ou Rotação RR
 - ▣ Rotação dupla a direita ou Rotação LR
 - ▣ Rotação dupla a esquerda ou Rotação RL

Rotações

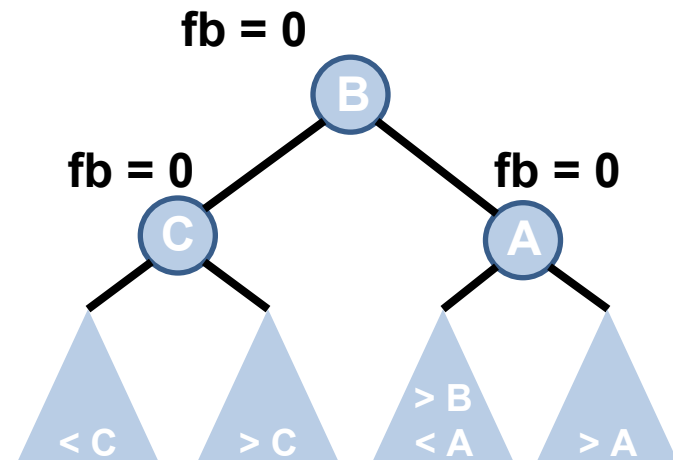
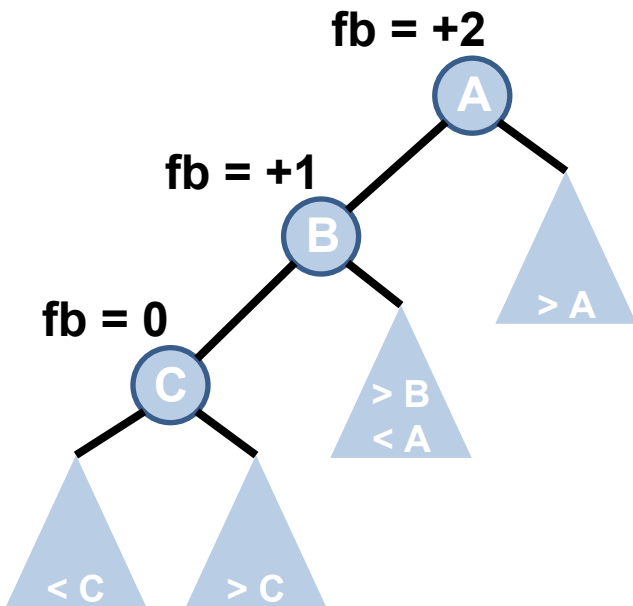
- Rotações são aplicadas no ancestral mais próximo do nó inserido cujo fator de balanceamento passa a ser $+2$ ou -2
- Após uma inserção ou remoção, devemos voltar pelo mesmo caminho da árvore e recalcular o fator de balanceamento, **fb**, de cada nó
- Se o **fb** desse nó for $+2$ ou -2 , uma rotação deverá ser aplicada

Rotação LL

- Rotação LL ou rotação simples à direita
 - ▣ Um novo nó é inserido na **sub-árvore da esquerda do filho esquerdo** de **A**
 - **A** é o nó desbalanceado
 - Dois movimentos para a esquerda: **LEFT LEFT**
 - ▣ É necessário fazer uma rotação à direita, de modo que o nó intermediário **B** ocupe o lugar de **A**, e **A** se torne a sub-árvore direita de **B**

Rotação LL

□ Exemplo



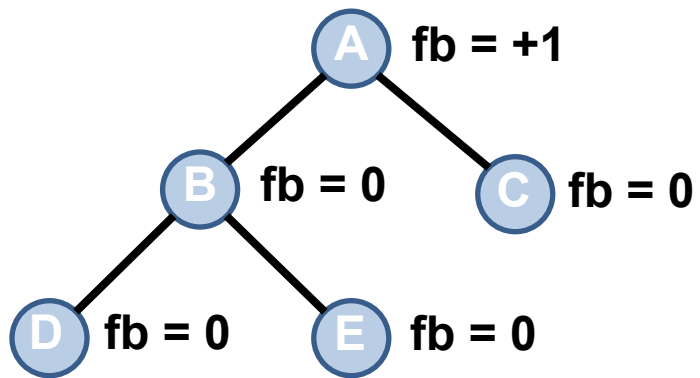
TAD Árvore AVL

□ Rotação LL

```
1
2 void RotacaoLL(ArvAVL *raiz){
3     struct NO *no;
4     no = (*raiz)->esq;
5     (*raiz)->esq = no->dir;
6     no->dir = *raiz;
7     (*raiz)->altura = maior(altura_NO((*raiz)->esq),
8                             altura_NO((*raiz)->dir))
9                             + 1;
10    no->altura = maior(altura_NO(no->esq),
11                      (*raiz)->altura) + 1;
12    *raiz = no;
13 }
```

Rotação LL

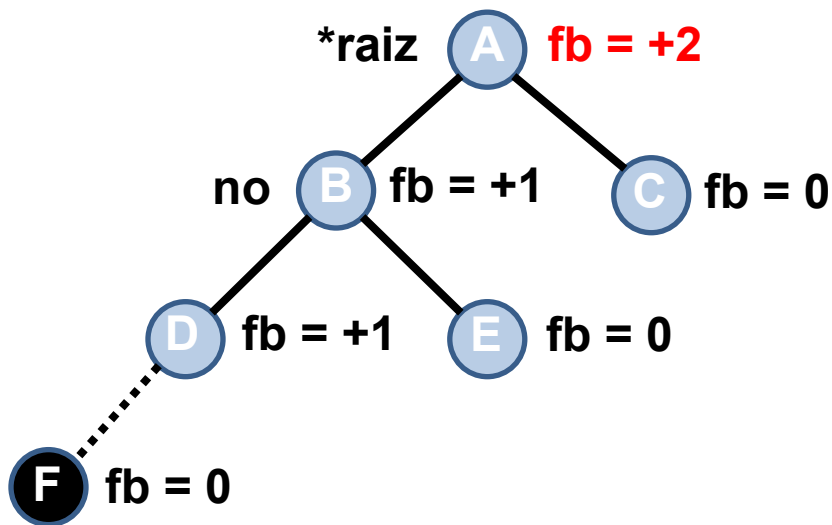
□ Passo a passo



Árvore AVL e fator de balanceamento de cada nó

Rotação LL

□ Passo a passo



Inserção do nó F na árvore

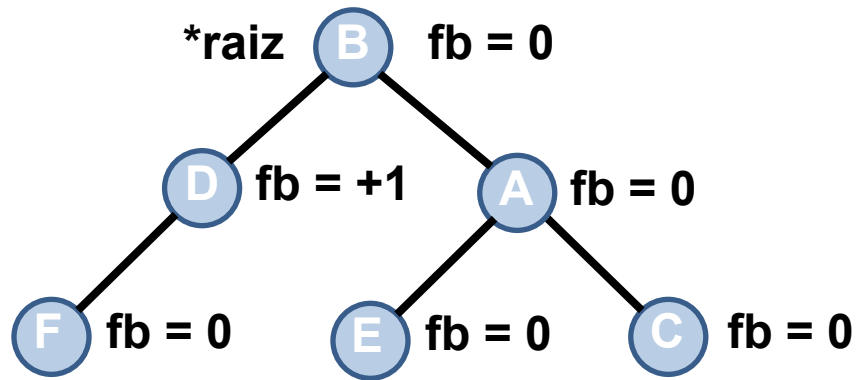
Árvore fica desbalanceada no nó A.

Aplicar Rotação LL no nó A

```
no = (*raiz)->esq;  
(*raiz)->esq = no->dir;  
no->dir = *raiz;  
*raiz = no;
```

Rotação LL

□ Passo a passo



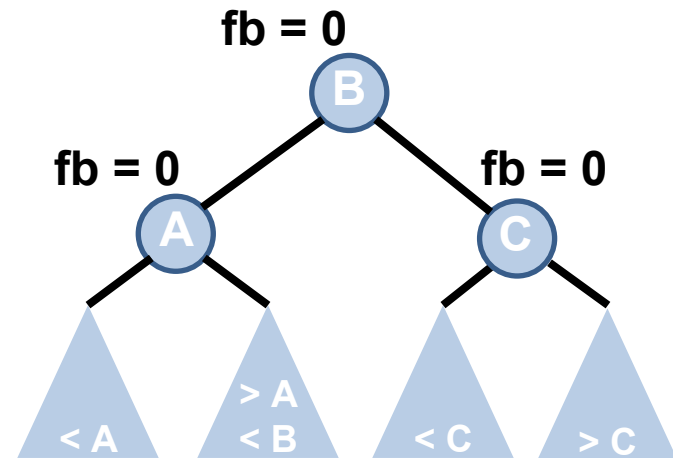
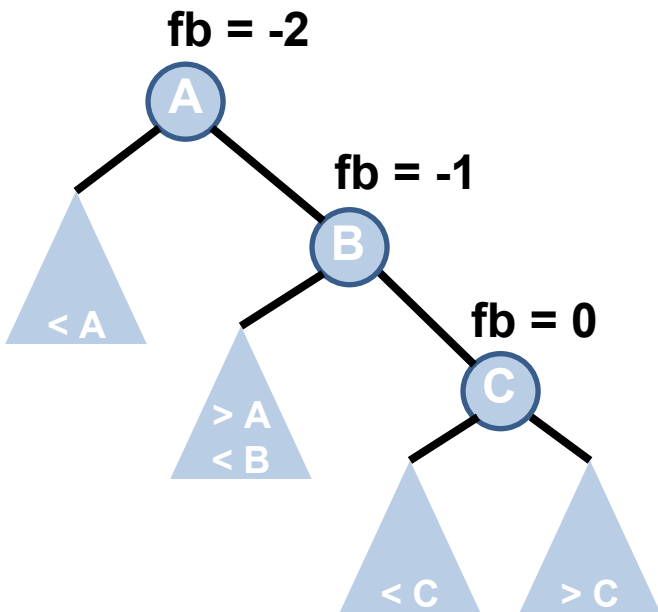
Árvore Balanceada

Rotação RR

- Rotação RR ou rotação simples à esquerda
 - ▣ Um novo nó é inserido na **sub-árvore da direita do filho direito de A**
 - **A** é o nó desbalanceado
 - Dois movimentos para a direita: **RIGHT RIGHT**
 - ▣ É necessário fazer uma rotação à esquerda, de modo que o nó intermediário **B** ocupe o lugar de **A**, e **A** se torne a sub-árvore esquerda de **B**

Rotação RR

Exemplo



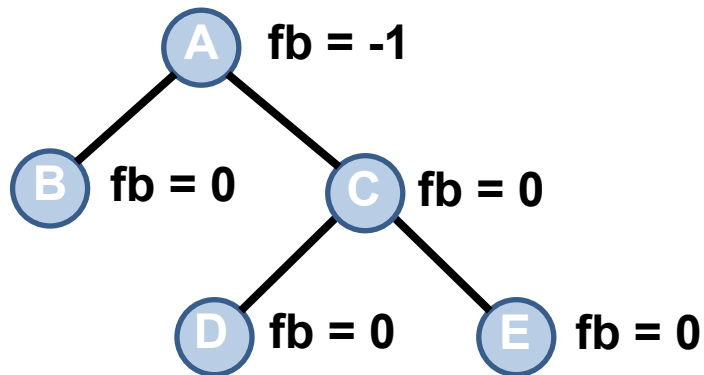
TAD Árvore AVL

□ Rotação RR

```
1
2 void RotacaoRR(ArvAVL *raiz){
3     struct NO *no;
4     no = (*raiz)->dir;
5     (*raiz)->dir = no->esq;
6     no->esq = (*raiz);
7     (*raiz)->altura = maior(altura_NO((*raiz)->esq),
8                             altura_NO((*raiz)->dir))
9                             + 1;
10    no->altura = maior(altura_NO(no->dir),
11                      (*raiz)->altura) + 1;
12    (*raiz) = no;
13 }
```

Rotação RR

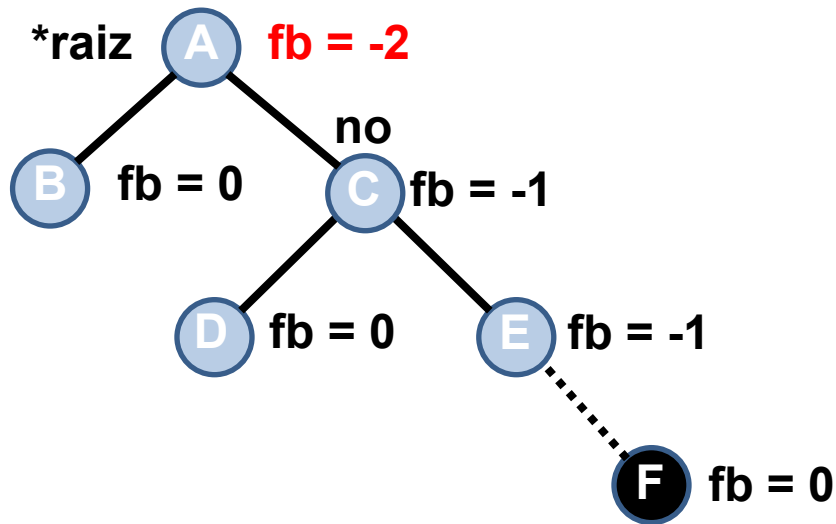
□ Passo a passo



Árvore AVL e fator de balanceamento de cada nó

Rotação RR

□ Passo a passo



Inserção do nó F na árvore

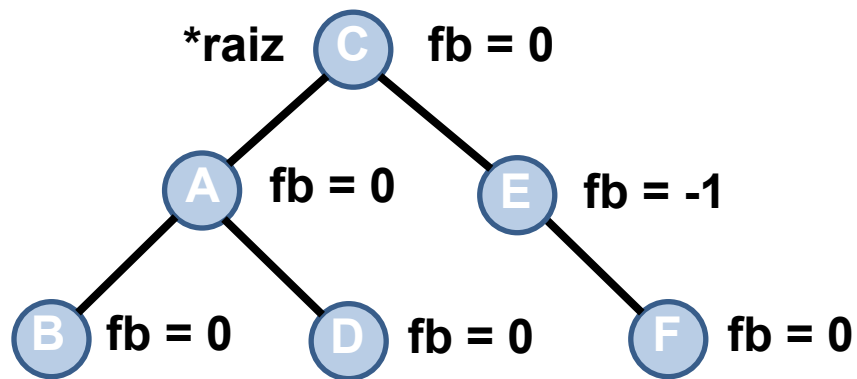
Árvore fica desbalanceada no nó A.

Aplicar Rotação RR no nó A

```
no = (*raiz)->dir;  
(*raiz)->dir = no->esq;  
no->esq = (*raiz);  
(*raiz) = no;
```

Rotação RR

□ Passo a passo



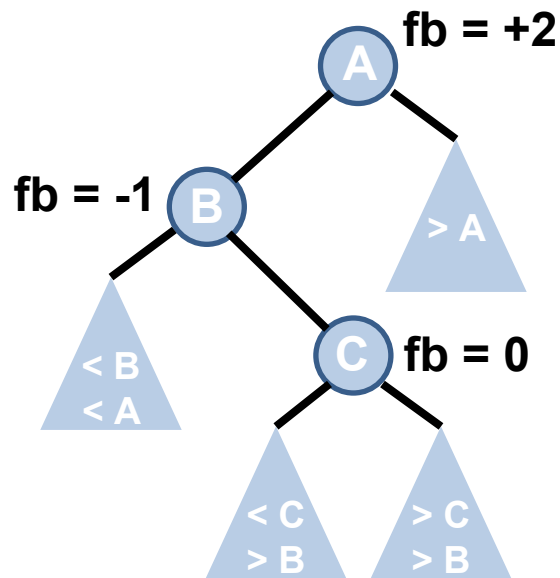
Árvore Balanceada

Rotação LR

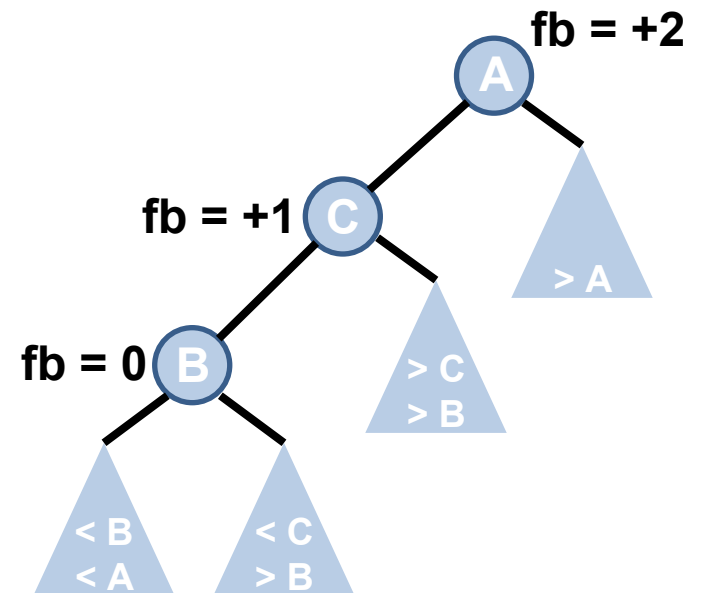
- Rotação LR ou rotação dupla à direita
 - ▣ Um novo nó é inserido na **sub-árvore da direita do filho esquerdo** de **A**
 - **A** é o nó desbalanceado
 - Um movimento para a esquerda e outro para a direita:
LEFT RIGHT
 - ▣ É necessário fazer uma rotação dupla, de modo que o nó **C** se torne o pai dos nós **A** (filho da direita) e **B** (filho da esquerda)
 - Rotação RR em **B**
 - Rotação LL em **A**

Rotação LR

□ Exemplo: primeira rotação



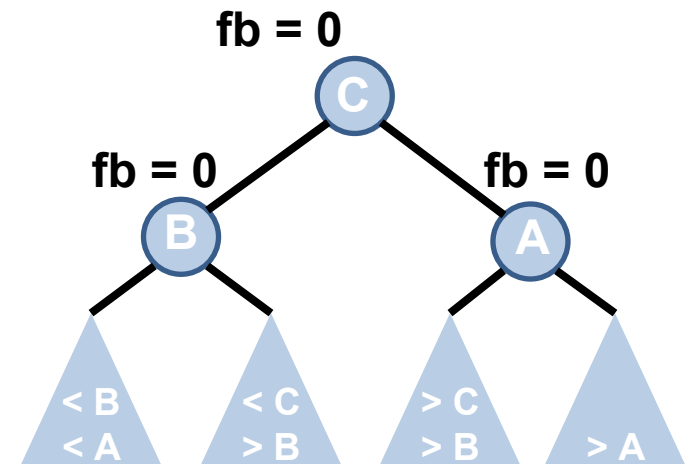
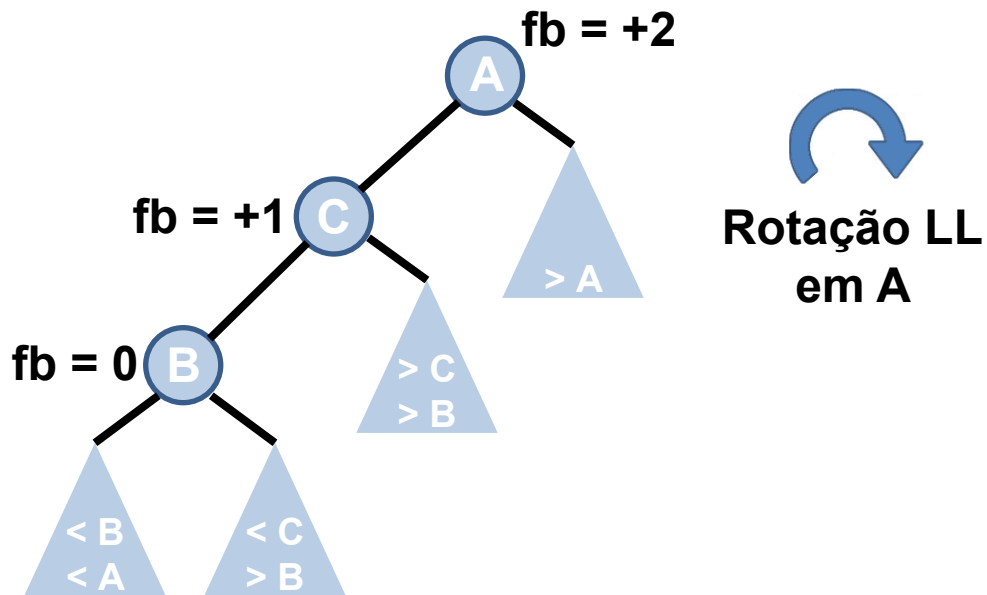
Rotação RR
em B



```
1
2 void RotacaoLR(ArvAVL *raiz) {
3     RotacaoRR(&(*raiz)->esq);
4     RotacaoLL(raiz);
5 }
```

Rotação LR

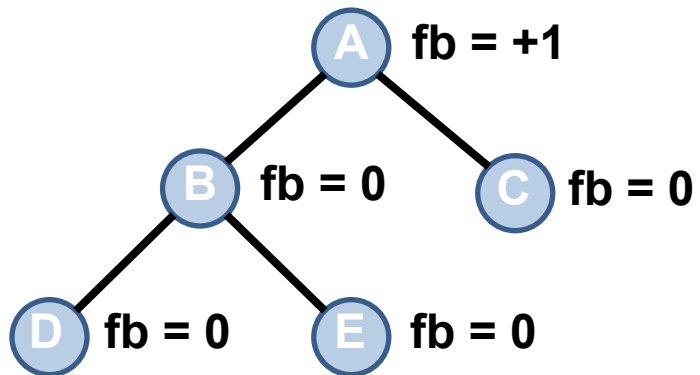
Exemplo: segunda rotação



```
1
2 void RotacaoLR(ArvAVL *raiz) {
3     RotacaoRR(&(*raiz)->esq);
4     RotacaoLL(raiz);
5 }
```

Rotação LR

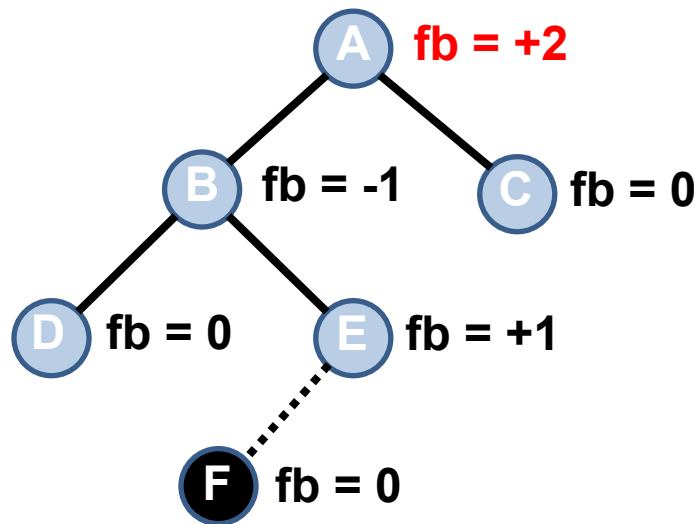
□ Passo a passo



Árvore AVL e fator de balanceamento de cada nó

Rotação LR

□ Passo a passo



Inserção do nó F na árvore

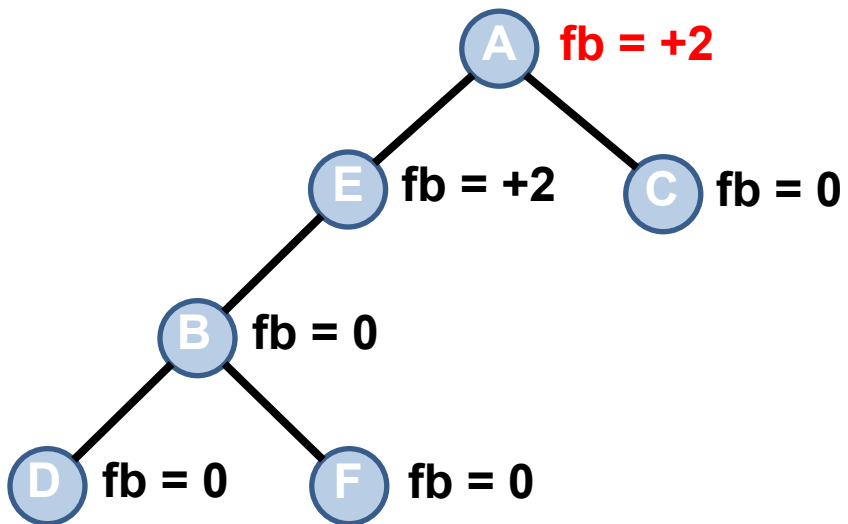
Árvore fica desbalanceada no nó A.

Aplicar Rotação LR no nó A.
Isso equivale a:

- Aplicar a Rotação RR no nó B
- Aplicar a Rotação LL no nó A

Rotação LR

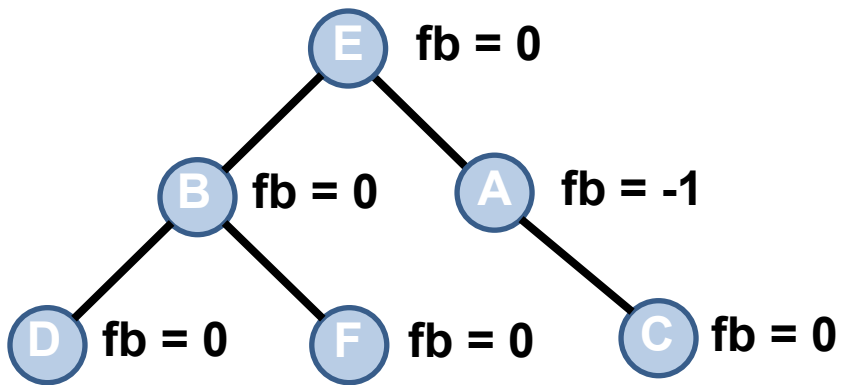
□ Passo a passo



Árvore após aplicar a Rotação RR no nó B

Rotação LR

□ Passo a passo



**Árvore após aplicar a
Rotação LL no nó A**

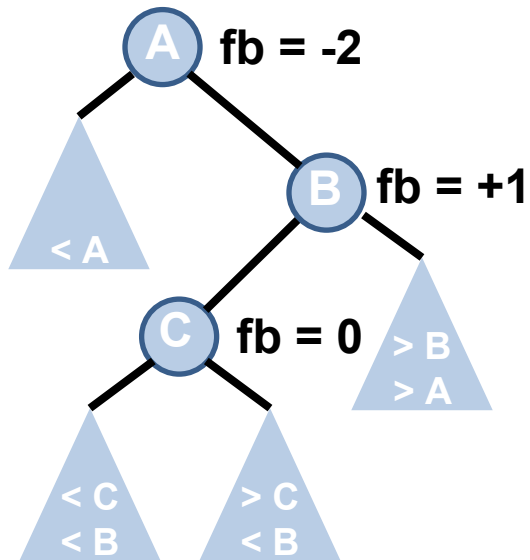
Árvore Balanceada

Rotação RL

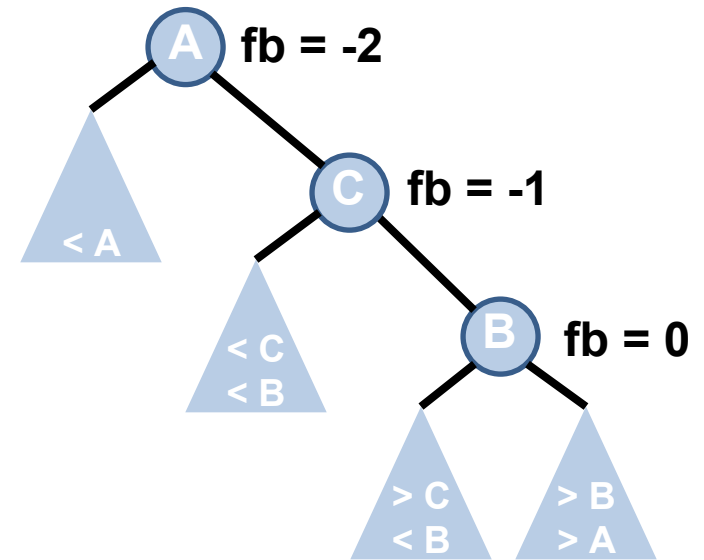
- Rotação RL ou rotação dupla à esquerda
 - ▣ um novo nó é inserido na **sub-árvore da esquerda do filho direito de A**
 - **A** é o nó desbalanceado
 - Um movimento para a direita e outro para a esquerda:
RIGHT LEFT
 - ▣ É necessário fazer uma rotação dupla, de modo que o nó **C** se torne o pai dos nós **A** (filho da esquerda) e **B** (filho da direita)
 - Rotação LL em **B**
 - Rotação RR em **A**

Rotação RL

□ Exemplo



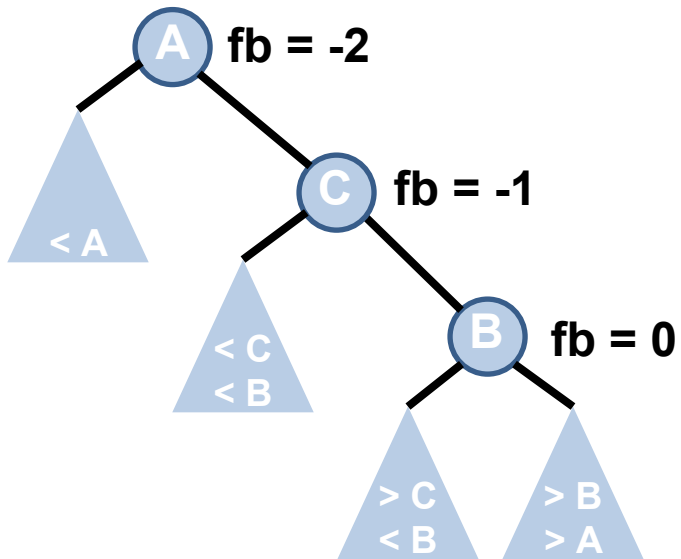

Rotação LL
em B



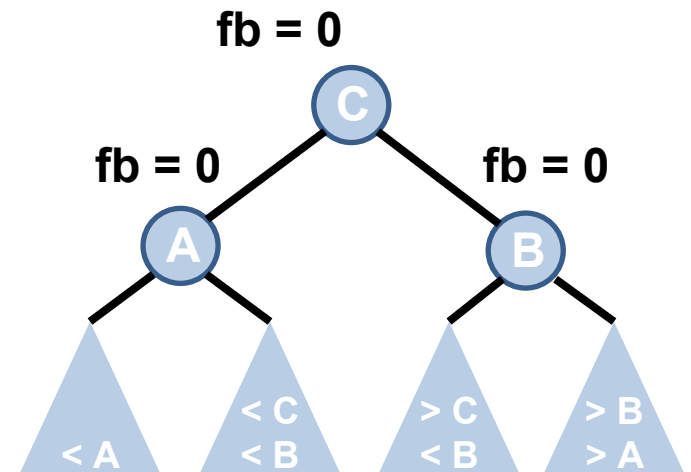
```
1
2 void RotacaoRL(ArvAVL *raiz) {
3     RotacaoLL(&(*raiz)->dir);
4     RotacaoRR(raiz);
5 }
```

Rotação RL

Exemplo



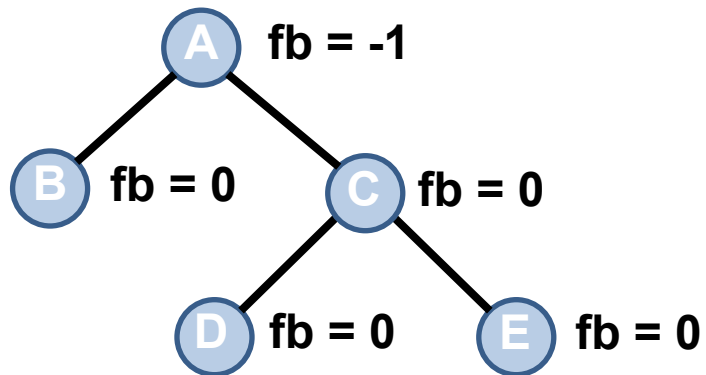
Rotação RR
em A



```
1  
2 void RotacaoRL(ArvAVL *raiz) {  
3     RotacaoLL(&(*raiz)->dir);  
4     RotacaoRR(raiz);  
5 }
```

Rotação RL

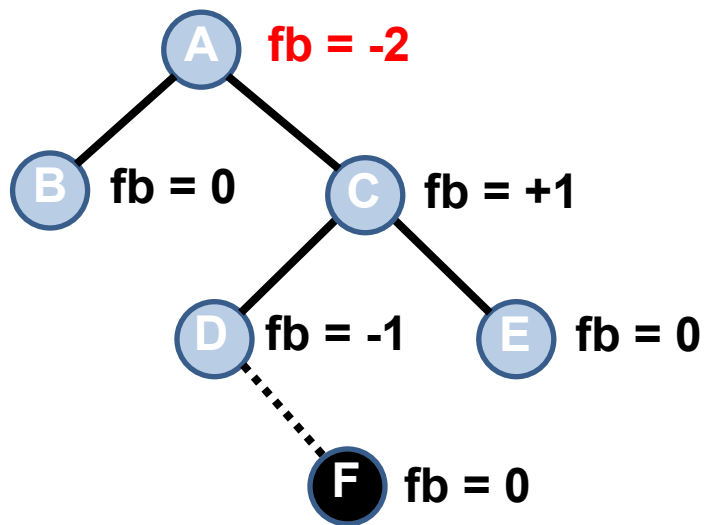
□ Passo a passo



Árvore AVL e fator de balanceamento de cada nó

Rotação RL

□ Passo a passo



Inserção do nó F na árvore

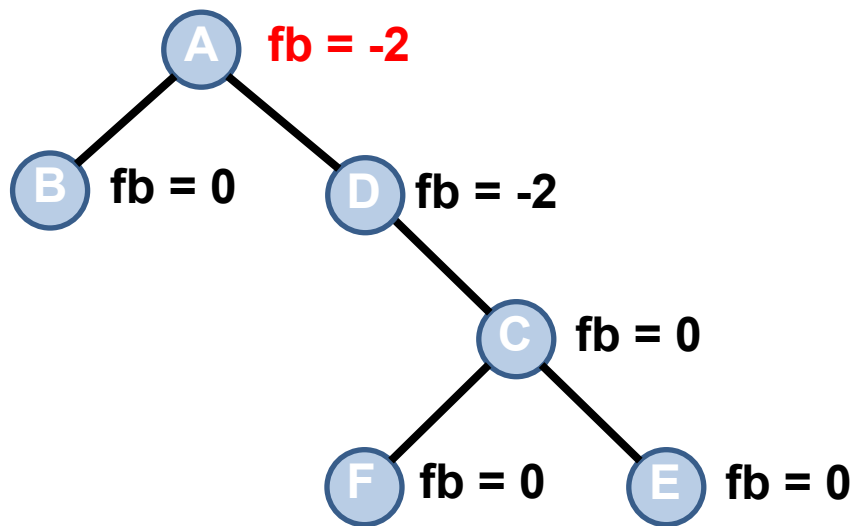
Árvore fica desbalanceada no nó A.

Aplicar Rotação RL no nó A.
Isso equivale a:

- Aplicar a Rotação LL no nó C
- Aplicar a Rotação RR no nó A

Rotação RL

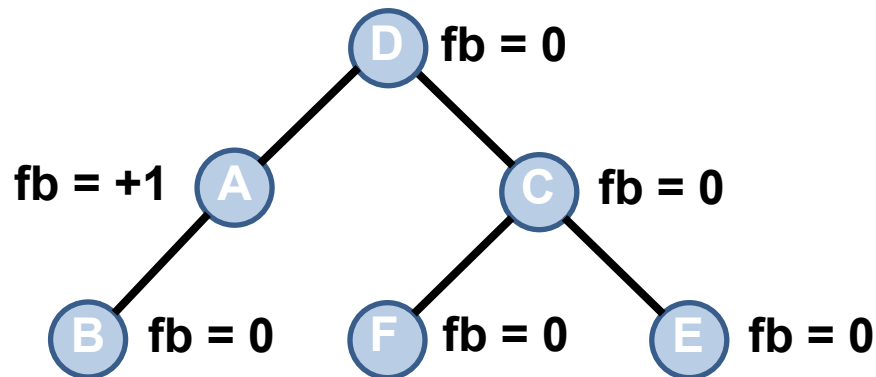
□ Passo a passo



Árvore após aplicar a Rotação LL no nó C

Rotação RL

□ Passo a passo



Árvore após aplicar a Rotação RR no nó A

Árvore Balanceada

Quando usar cada rotação?

- Uma dúvida muito comum é quando utilizar cada uma das quatro rotações

Fator de Balanceamento de A	Fator de Balanceamento de B	Posições dos nós B e C em relação ao nó A	Rotação
+2	+1	B é filho à esquerda de A C é filho à esquerda de B	LL
-2	-1	B é filho à direita de A C é filho à direita de B	RR
+2	-1	B é filho à esquerda de A C é filho à direita de B	LR
-2	+1	B é filho à de direita A C é filho à esquerda de B	RL

Quando usar cada rotação?

- Sinais iguais: rotação simples
 - ▣ Sinal positivo: rotação à direita (LL)
 - ▣ Sinal negativo: rotação à esquerda (RR)

Fator de Balanceamento de A	Fator de Balanceamento de B	Posições dos nós B e C em relação ao nó A	Rotação
+2	+1	B é filho à esquerda de A C é filho à esquerda de B	LL
-2	-1	B é filho à direita de A C é filho à direita de B	RR
+2	-1	B é filho à esquerda de A C é filho à direita de B	LR
-2	+1	B é filho à de direita A C é filho à esquerda de B	RL

Quando usar cada rotação?

- Sinais diferentes: rotação dupla
 - ▣ **A** positivo: rotação dupla a direita (LR)
 - ▣ **A** negativo: rotação dupla a esquerda (RL)

Fator de Balanceamento de A	Fator de Balanceamento de B	Posições dos nós B e C em relação ao nó A	Rotação
+2	+1	B é filho à esquerda de A C é filho à esquerda de B	LL
-2	-1	B é filho à direita de A C é filho à direita de B	RR
+2	-1	B é filho à esquerda de A C é filho à direita de B	LR
-2	+1	B é filho à de direita A C é filho à esquerda de B	RL

Árvore AVL: Inserção

- Para inserir um valor **V** na árvore
 - ▣ Se a raiz é igual a **NULL**, insira o nó
 - ▣ Se **V** é menor do que a raiz: vá para a **sub-árvore esquerda**
 - ▣ Se **V** é maior do que a raiz: vá para a **sub-árvore direita**
 - ▣ Aplique o método **recursivamente**
- Dessa forma, percorremos um conjunto de nós da árvore até chegar ao nó folha que irá se tornar o pai do novo nó

Árvore AVL: Inserção

- Uma vez inserido o novo nó
 - ▣ Devemos voltar pelo caminho percorrido e calcular o fator de balanceamento de cada um dos nós visitados
 - ▣ Aplicar a rotação necessária para restabelecer o balanceamento da árvore se o fator de balanceamento for **+2** ou **-2**

TAD Árvore AVL

□ Inserção

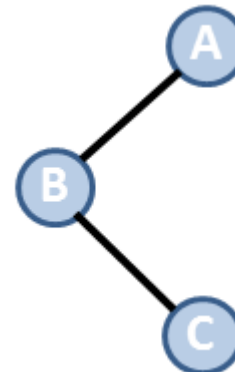
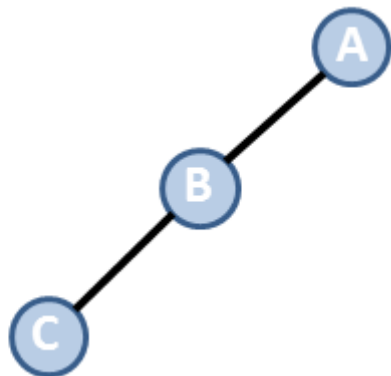
```
int insere_ArvAVL(ArvAVL *raiz, int valor){
    int res;
    if(*raiz == NULL){//árvore vazia ou nó folha
        struct NO *novo;
        novo = (struct NO*)malloc(sizeof(struct NO));
        if(novo == NULL)
            return 0;

        novo->info = valor;
        novo->altura = 0;
        novo->esq = NULL;
        novo->dir = NULL;
        *raiz = novo;
        return 1;
    }
    //continua...
```

TAD Árvore AVL

□ Inserção

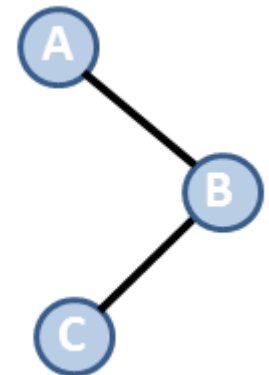
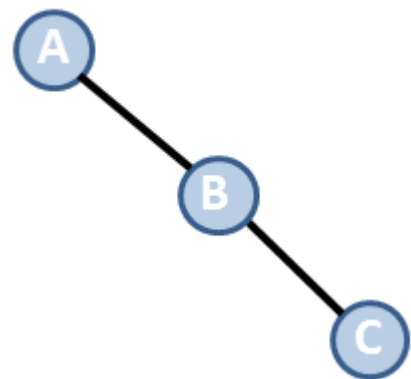
```
//continuação
struct NO *atual = *raiz;
if(valor < atual->info){
    if((res=insere_ArvAVL(&(atual->esq), valor))==1){
        if(fatorBalanceamento_NO(atual) >= 2){
            if(valor < (*raiz)->esq->info ){
                RotacaoLL(raiz);
            } else{
                RotacaoLR(raiz);
            }
        }
    }
}
```



TAD Árvore AVL

□ Inserção

```
//continuação
else{
    if(valor > atual->info){
        if((res=inserere_ArvAVL(&(atual->dir), valor))==1){
            if(fatorBalanceamento_NO(atual) >= 2){
                if((*raiz)->dir->info < valor){
                    RotacaoRR(raiz);
                }else{
                    RotacaoRL(raiz);
                }
            }
        }else{
            printf("Valor duplicado!!\n");
            return 0;
        }
    }
    atual->altura = maior(altura_NO(atual->esq),
                        altura_NO(atual->dir)) + 1;
    return res;
}
```



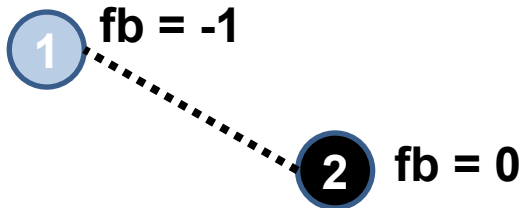
Árvore AVL: Inserção

□ Passo a passo

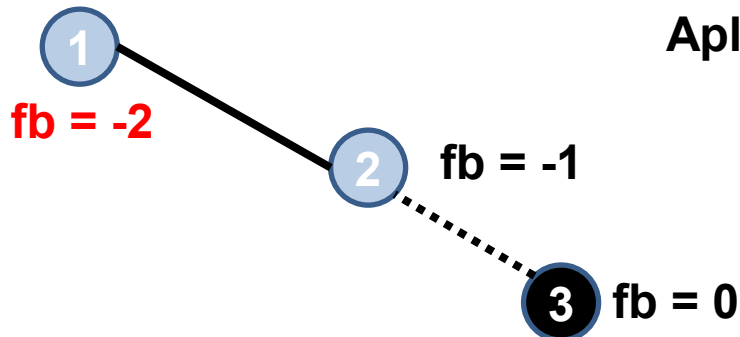
Inserir valor: 1



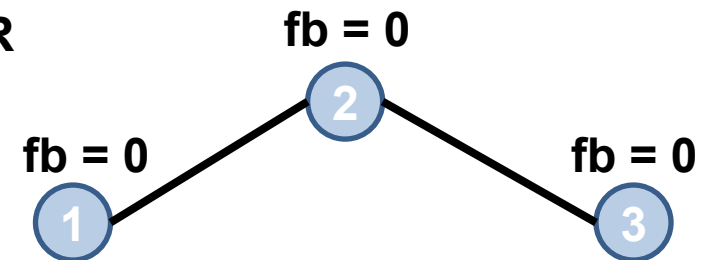
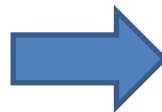
Inserir valor: 2



Inserir valor: 3



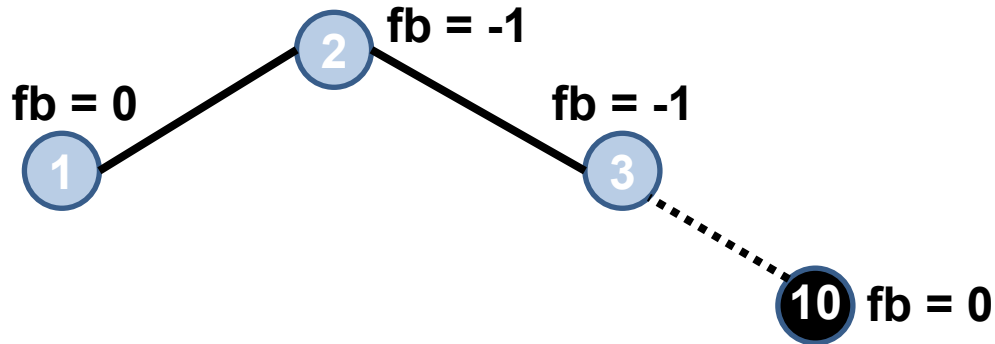
Nó "1" desbalanceado
Aplicar Rotação RR



Árvore AVL: Inserção

□ Passo a passo

Inserir valor: 10

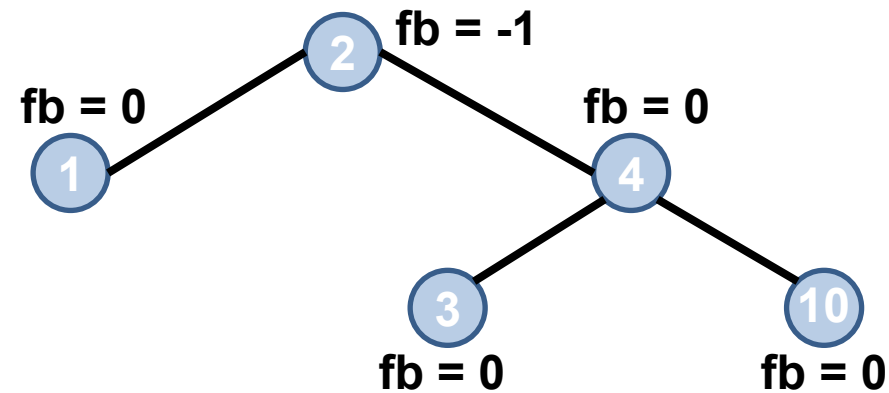
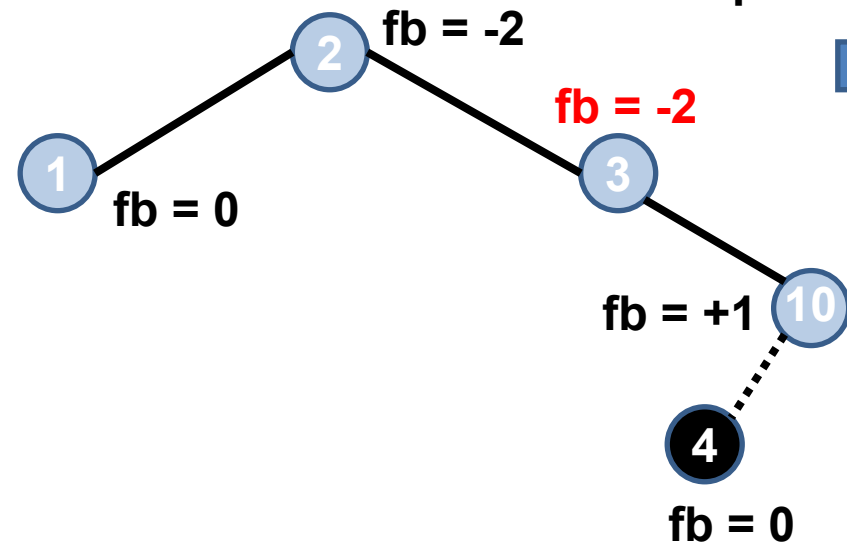
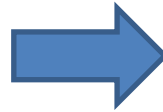


Árvore AVL: Inserção

□ Passo a passo

Inserir valor: 4

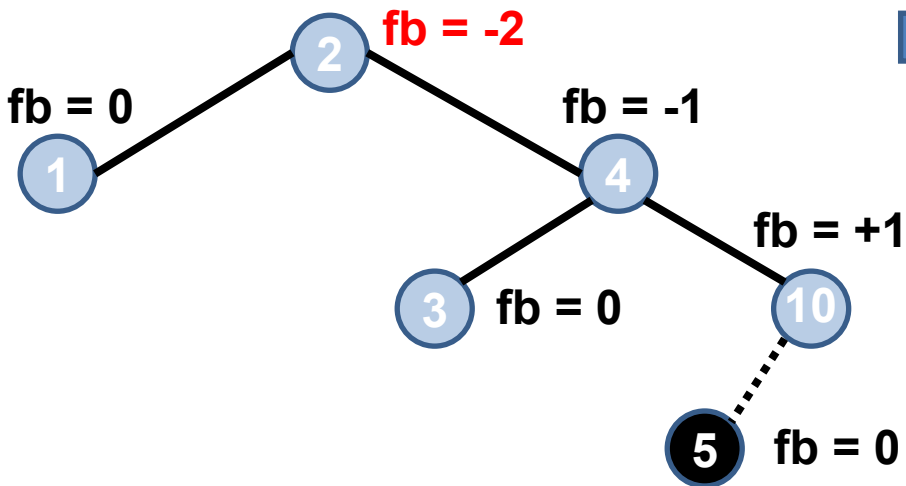
Nó "3" desbalanceado
Aplicar Rotação RL



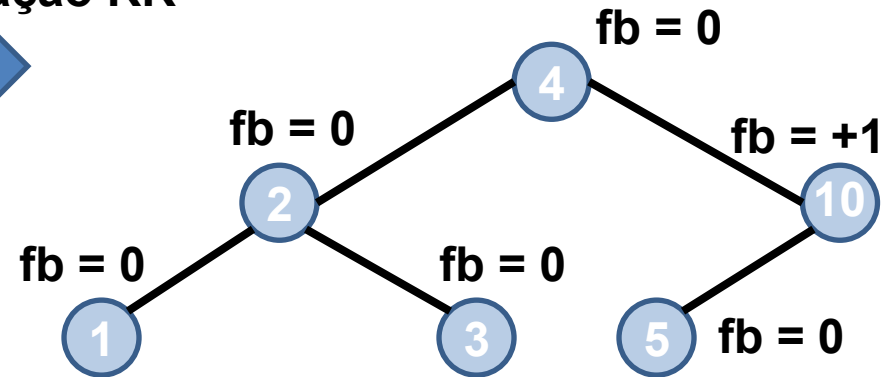
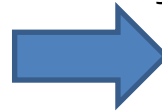
Árvore AVL: Inserção

□ Passo a passo

Inserir valor: 5



Nó "2" desbalanceado
Aplicar Rotação RR

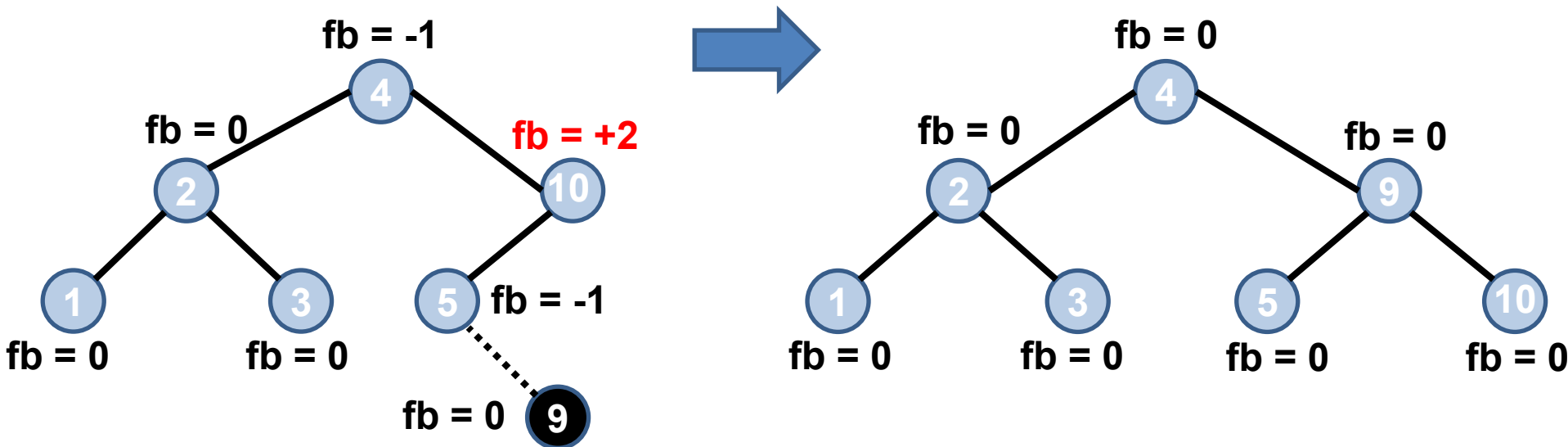


Árvore AVL: Inserção

□ Passo a passo

Inserir valor: 9

Nó "10" desbalanceado
Aplicar Rotação LR



Árvore AVL: Remoção

- Como na inserção, temos que percorrer um conjunto de nós da árvore até chegarmos ao nó que será removido
 - ▣ Existem 3 tipos de remoção
 - Nó folha (sem filhos)
 - Nó com 1 filho
 - Nó com 2 filhos

Árvore AVL: Remoção

- Uma vez removido o nó
 - ▣ Devemos voltar pelo caminho percorrido e calcular o fator de balanceamento de cada um dos nós visitados
 - ▣ Aplicar a rotação necessária para restabelecer o balanceamento da árvore se o fator de balanceamento for **+2** ou **-2**
 - **Remover** um nó da sub-árvore **direita** equivale a **inserir** um nó na sub-árvore **esquerda**

TAD Árvore AVL

□ Remoção

▣ Trabalha com 2 funções

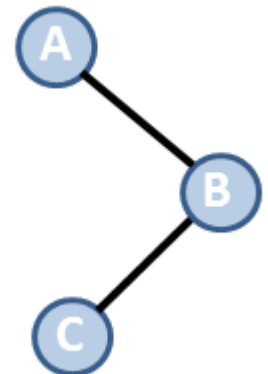
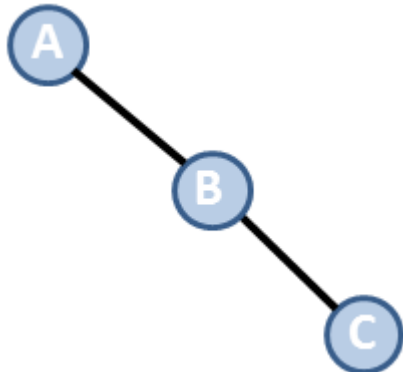
- Busca pelo nó
- Remoção do nó com 2 filhos

```
8  int remove_ArvAVL(ArvAVL *raiz, int valor){
9      /*
10     FUNÇÃO RESPONSÁVEL PELA BUSCA
11     DO NÓ A SER REMOVIDO
12     */
13 }
14 struct NO* procuraMenor(struct NO* atual){
15     /*
16     FUNÇÃO RESPONSÁVEL POR TRATAR OS
17     A REMOÇÃO DE UM NÓ COM 2 FILHOS
18     */
19 }
```

TAD Árvore AVL

□ Remoção

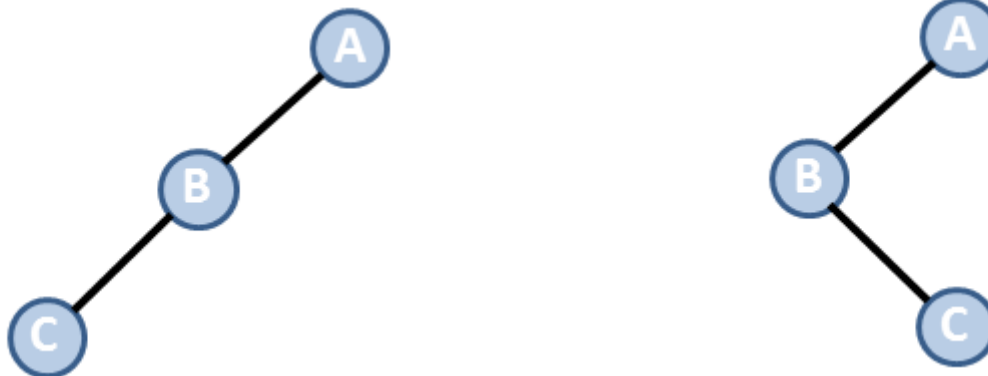
```
int remove_ArvAVL(ArvAVL *raiz, int valor){
    if(*raiz == NULL){// valor não existe
        printf("valor não existe!!\n");
        return 0;
    }
    int res;
    if(valor < (*raiz)->info){
        if((res=remove_ArvAVL(&(*raiz)->esq, valor))==1)
            if(fatorBalanceamento_NO(*raiz) >= 2){
                if(altura_NO((*raiz)->dir->esq)
                    <= altura_NO((*raiz)->dir->dir))
                    RotacaoRR(raiz);
                else
                    RotacaoRL(raiz);
            }
    }
    //continua...
```



TAD Árvore AVL

□ Remoção

```
//continuação...  
if ((*raiz)->info < valor) {  
    if ((res=remove_ArvAVL(&(*raiz)->dir, valor))==1) {  
        if (fatorBalanceamento_NO(*raiz) >= 2) {  
            if (altura_NO((*raiz)->esq->dir)  
                <= altura_NO((*raiz)->esq->esq) )  
                RotacaoLL(raiz);  
            else  
                RotacaoLR(raiz);  
        }  
    }  
}  
//continua...
```



TAD Árvore AVL

□ Remoção

Pai tem 1 ou
nenhum filho

```
if ((*raiz)->info == valor){  
    if (((*raiz)->esq == NULL || (*raiz)->dir == NULL)) { // nó tem 1 filho ou nenhum  
        struct NO *oldNode = (*raiz);  
        if ((*raiz)->esq != NULL)  
            *raiz = (*raiz)->esq;  
        else  
            *raiz = (*raiz)->dir;  
        free(oldNode);  
    } else { // nó tem 2 filhos
```

Pai tem 2 filhos:
Substituir pelo nó
mais a esquerda
da sub-árvore da
direita

```
        struct NO* temp = procuraMenor((*raiz)->dir);  
        (*raiz)->info = temp->info;  
        remove_ArvAVL(&(*raiz)->dir, (*raiz)->info);  
        if (fatorBalanceamento_NO(*raiz) >= 2){  
            if (altura_NO((*raiz)->esq->dir) <= altura_NO((*raiz)->esq->esq))  
                RotacaoLL(raiz);  
            else  
                RotacaoLR(raiz);  
        }  
    }
```

Corrige a
altura

```
    if (*raiz != NULL)  
        (*raiz)->altura = maior(altura_NO((*raiz)->esq),  
                                altura_NO((*raiz)->dir)) + 1;  
  
    return 1;  
}  
  
(*raiz)->altura = maior(altura_NO((*raiz)->esq),  
                        altura_NO((*raiz)->dir)) + 1;
```

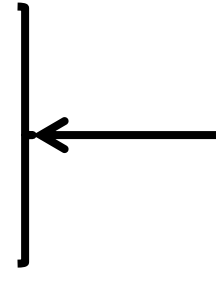
```
return res;
```

```
}
```

TAD Árvore AVL

□ Remoção

```
struct NO* procuraMenor(struct NO* atual) {  
    struct NO *no1 = atual;  
    struct NO *no2 = atual->esq;  
    while(no2 != NULL) {  
        no1 = no2;  
        no2 = no2->esq;  
    }  
    return no1;  
}
```

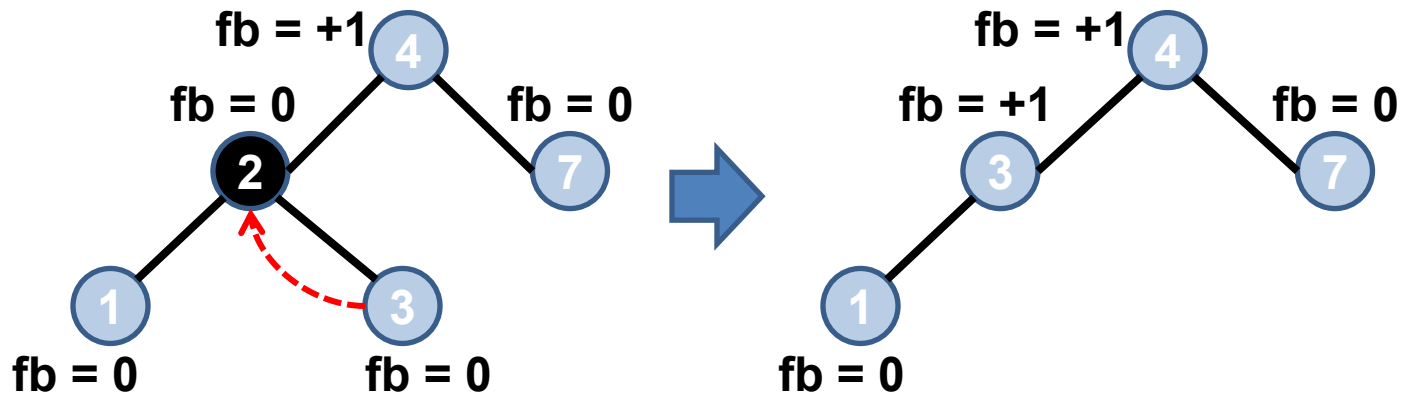


Procura pelo nó
mais a esquerda

Árvore AVL: Remoção

□ Passo a passo

Remove valor: 2



Árvore AVL: Remoção

□ Passo a passo

Remove valor: 7

